

GUÍA DE LABORATORIO 07

JPQL Y TRANSACCIONES CON SPRING BOOT

Consultas personalizadas y operaciones atómicas con Spring Data JPA

CURSO	SEMANA	DURACIÓN	MODALIDAD
Desarrollo Web Integrado	7	3 horas	Práctica guiada

CONTINUIDAD PEDAGÓGICA

Semana 6: Entity + Repository + CRUD + MySQL → Semana 7: JPQL + @Transactional + rollback + reglas de negocio.

STACK DE TRABAJO

Java 25 | Spring Boot | Maven | JPA/Hibernate | MySQL | VS Code | Postman

Resultado esperado

El estudiante implementa consultas JPQL y delimita transacciones de negocio capaces de confirmar cambios (commit) o revertirlos (rollback) de forma consistente.

1. Propósito y logro de aprendizaje

De acuerdo con el sílabo, la Semana 7 corresponde a “JPQL. Transacciones con Spring Boot”. Esta práctica amplía el CRUD de la Semana 6 para que el estudiante pueda resolver consultas que no se cubren cómodamente con los métodos heredados de JpaRepository y, además, controlar operaciones de negocio que deben ejecutarse como una sola unidad.

LOGRO DEL LABORATORIO

Al finalizar la práctica, el estudiante implementa consultas personalizadas con JPQL y gestiona transacciones con Spring Boot para asegurar consistencia, commit y rollback sobre una base de datos MySQL.

1.1. Capacidades que se desarrollan

- Diferenciar SQL, JPQL y consultas derivadas de Spring Data JPA.
- Construir consultas JPQL con @Query y parámetros nombrados.
- Aplicar @Transactional en la capa de servicio, donde reside la lógica de negocio.
- Comprender commit, rollback, atomicidad y el contexto de persistencia.
- Comprobar el comportamiento transaccional mediante una operación de inventario.
- Probar endpoints con Postman y verificar el efecto real en MySQL.

1.2. Prerrequisitos

Requisito	Verificación rápida
Proyecto de Semana 6	La API CRUD de productos debe iniciar sin errores.
Java	java --version → Java 25
Maven	mvn -version
MySQL	Base de datos disponible y credenciales válidas.
VS Code	Extension Pack for Java + Spring Boot Extension Pack.
Postman	Disponible para ejecutar las pruebas HTTP.

ANTES DE MODIFICAR EL PROYECTO

Haz una copia o commit del proyecto de la Semana 6. La Semana 7 debe permitir comparar claramente el antes (CRUD) y el después (consultas especializadas + transacciones).

2. Marco conceptual mínimo

2.1. ¿Dónde encaja JPQL?

Spring Data JPA ofrece varios niveles para consultar datos. Conviene usar el nivel más simple que resuelva la necesidad y reservar JPQL para consultas que requieran mayor expresividad.

Nivel	Ejemplo	Cuándo usarlo
CRUD heredado	findAll(), findById(), save()	Operaciones estándar.
Consulta derivada	findByCategoryIgnoreCase(...)	Filtros simples expresables por nombre de método.
JPQL con @Query	SELECT p FROM Producto p WHERE ...	Consultas personalizadas sobre entidades y atributos.
SQL nativo	nativeQuery = true	Solo cuando una característica específica del motor lo justifique.

2.2. JPQL no consulta tablas: consulta entidades

JPQL (Java Persistence Query Language) se parece a SQL, pero trabaja con nombres de clases y atributos Java administrados por JPA. Hibernate traduce la consulta JPQL al SQL correspondiente para MySQL.

SQL	JPQL
SELECT * FROM productos WHERE precio > 100	SELECT p FROM Producto p WHERE p.precio > :minimo
productos = tabla	Producto = entidad Java
precio = columna	p.precio = atributo de la entidad

IDEA CLAVE

JPQL mantiene la consulta en el modelo de objetos de la aplicación. No conviene escribir nombres físicos de tablas o columnas dentro de JPQL.

2.3. ¿Qué es una transacción?

Una transacción es una unidad de trabajo: todas las operaciones que forman parte de ella se confirman juntas o se revierten juntas. En Spring, la frontera transaccional suele declararse en la capa Service con `@Transactional`.

Propiedad ACID	Idea práctica
Atomicidad	Todo o nada: si una parte falla, se revierte la operación completa.
Consistencia	La base de datos pasa de un estado válido a otro estado válido.
Aislamiento	Las transacciones concurrentes no deberían observar estados intermedios inválidos.
Durabilidad	Una vez confirmado el commit, el cambio persiste.

2.4. Commit, rollback y dirty checking

Dentro de una transacción, una entidad cargada desde un repositorio queda administrada por el contexto de persistencia. Si se modifica un atributo, Hibernate puede detectar el cambio y emitir el UPDATE al confirmar la transacción. Este comportamiento se denomina dirty checking.

Ejemplo conceptual

```
@Transactional
public Producto cambiarPrecio(Long id, BigDecimal nuevoPrecio) {
    Producto producto = productoRepository.findById(id)
        .orElseThrow(() -> new RecursoNoEncontradoException("Producto no encontrado"));

    producto.setPrecio(nuevoPrecio);

    // No es obligatorio llamar save(producto) si la entidad continúa administrada.
    return producto;
}
```

ROLLBACK POR DEFECTO

Spring revierte una transacción cuando se propaga una RuntimeException (o Error). Para excepciones checked, el comportamiento debe configurarse explícitamente si se requiere rollback.

2.5. Arquitectura que utilizaremos



La transacción se define en Service porque allí se coordinan reglas de negocio y, potencialmente, múltiples repositorios. El Controller se limita a HTTP; el Repository se limita al acceso a datos.

3. Caso práctico: inventario de productos

Continuaremos con la API de productos de la Semana 6. Se incorporarán búsquedas avanzadas y una operación de salida de inventario. Para demostrar atomicidad, cada salida actualizará el stock y registrará un movimiento dentro de la misma transacción.

3.1. Endpoints que debe ofrecer la API

Método	Endpoint	Objetivo
GET	/api/productos/buscar?texto=lap	Buscar por parte del nombre.
GET	/api/productos/categoria/{categoria}	Filtrar por categoría.
GET	/api/productos/precio? min=100&max=1000	Buscar por rango de precio.
POST	/api/productos/{id}/salidas?cantidad=2	Descontar stock y registrar movimiento.
POST	/api/productos/{id}/salidas/simular-error? cantidad=1	Forzar error y comprobar rollback.

3.2. Estructura esperada del proyecto

```
src/main/java/com/utp/tienda/
├── controller/
│   └── ProductoController.java
├── model/
│   ├── Producto.java
│   └── MovimientoStock.java
├── repository/
│   ├── ProductoRepository.java
│   └── MovimientoStockRepository.java
├── service/
│   └── ProductoService.java
├── exception/
│   ├── RecursoNoEncontradoException.java
│   ├── ReglaNegocioException.java
│   └── ApiExceptionHandler.java
└── TiendaApplication.java
```

4. Paso 1 - Verificar la configuración de MySQL y JPA

Conserve la configuración funcional de la Semana 6. Un ejemplo de referencia es el siguiente:

src/main/resources/application.properties

```
spring.application.name=tienda

spring.datasource.url=jdbc:mysql://localhost:3306/tienda_db?useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=TU_CLAVE

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

SEGURIDAD DE CONFIGURACIÓN

No subas credenciales reales a un repositorio remoto. En un entorno profesional se usan variables de entorno o mecanismos de secretos. Para este laboratorio local, la prioridad es comprender la persistencia y las transacciones.

4.1. Verificar dependencias Maven

El proyecto debe incluir, como mínimo, Spring Web, Spring Data JPA, MySQL Driver y Spring Boot Starter Test. No agregues Hibernate manualmente: Spring Data JPA gestiona la implementación ORM compatible.

Dependencias relevantes del pom.xml

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>

```

5. Paso 2 - Consolidar la entidad Producto

La entidad debe contener información suficiente para practicar filtros y reglas de inventario. Si tu clase de la Semana 6 ya existe, compara los campos y adapta solo lo necesario.

src/main/java/com/utp/tienda/model/Producto.java

```

package com.utp.tienda.model;

import java.math.BigDecimal;
import jakarta.persistence.*;

@Entity
@Table(name = "productos")
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120)
    private String nombre;

    @Column(nullable = false, length = 80)
    private String categoria;

    @Column(nullable = false, precision = 12, scale = 2)
    private BigDecimal precio;

    @Column(nullable = false)
    private Integer stock;

    public Producto() {}

    public Producto(String nombre, String categoria, BigDecimal precio, Integer stock) {
        this.nombre = nombre;
        this.categoria = categoria;
        this.precio = precio;
        this.stock = stock;
    }

    public Long getId() { return id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getCategoria() { return categoria; }
    public void setCategoria(String categoria) { this.categoria = categoria; }
    public BigDecimal getPrecio() { return precio; }
    public void setPrecio(BigDecimal precio) { this.precio = precio; }
    public Integer getStock() { return stock; }
    public void setStock(Integer stock) { this.stock = stock; }
}

```

¿POR QUÉ BigDecimal?

Para importes monetarios, BigDecimal evita muchos problemas de precisión binaria asociados a double. Es una decisión de modelado más apropiada para precios.

6. Paso 3 - Crear la entidad MovimientoStock

El movimiento permite demostrar una transacción que coordina dos cambios: actualizar el producto y registrar la evidencia de la salida.

src/main/java/com/utp/tienda/model/MovimientoStock.java

```
package com.utp.tienda.model;

import java.time.LocalDateTime;
import jakarta.persistence.*;

@Entity
@Table(name = "movimientos_stock")
public class MovimientoStock {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "producto_id", nullable = false)
    private Producto producto;

    @Column(nullable = false, length = 20)
    private String tipo;

    @Column(nullable = false)
    private Integer cantidad;

    @Column(nullable = false)
    private LocalDateTime fecha;

    public MovimientoStock() {}

    public MovimientoStock(Producto producto, String tipo, Integer cantidad, LocalDateTime fecha) {
        this.producto = producto;
        this.tipo = tipo;
        this.cantidad = cantidad;
        this.fecha = fecha;
    }

    public Long getId() { return id; }
    public Producto getProducto() { return producto; }
    public String getTipo() { return tipo; }
    public Integer getCantidad() { return cantidad; }
    public LocalDateTime getFecha() { return fecha; }
}
```

7. Paso 4 - Implementar consultas derivadas y JPQL

Ahora se amplía ProductoRepository. Observe que las consultas JPQL usan Producto, p.nombre, p.precio y p.stock: todos son elementos del modelo Java, no nombres de tablas SQL.

src/main/java/com/utp/tienda/repository/ProductoRepository.java

```
package com.utp.tienda.repository;

import java.math.BigDecimal;
import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import com.utp.tienda.model.Producto;

public interface ProductoRepository extends JpaRepository<Producto, Long> {

    // Consulta derivada: Spring Data interpreta el nombre del método.
    List<Producto> findByCategoriaIgnoreCase(String categoria);

    // JPQL: búsqueda parcial sin distinguir mayúsculas/minúsculas.
    @Query("""
```

```

        SELECT p
        FROM Producto p
        WHERE LOWER(p.nombre) LIKE LOWER(CONCAT('%', :texto, '%'))
        ORDER BY p.nombre
        """)
    List<Producto> buscarPorNombre(@Param("texto") String texto);

    // JPQL: rango de precios.
    @Query("""
        SELECT p
        FROM Producto p
        WHERE p.precio BETWEEN :min AND :max
        ORDER BY p.precio ASC
        """)
    List<Producto> buscarPorRangoPrecio(
        @Param("min") BigDecimal min,
        @Param("max") BigDecimal max);

    // JPQL adicional: productos con stock bajo.
    @Query("SELECT p FROM Producto p WHERE p.stock <= :limite ORDER BY p.stock ASC")
    List<Producto> buscarConStockBajo(@Param("limite") Integer limite);
}

```

src/main/java/com/utp/tienda/repository/MovimientoStockRepository.java

```

package com.utp.tienda.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.utp.tienda.model.MovimientoStock;

public interface MovimientoStockRepository
    extends JpaRepository<MovimientoStock, Long> {
}

```

PREGUNTA DE CONTROL

¿Por qué buscar por categoría no necesita @Query? Porque la consulta derivada findByCategorialgnoreCase ya expresa claramente la intención. JPQL se usa cuando aporta expresividad adicional.

8. Paso 5 - Crear excepciones de negocio

RecursoNoEncontradoException.java

```

package com.utp.tienda.exception;

public class RecursoNoEncontradoException extends RuntimeException {
    public RecursoNoEncontradoException(String mensaje) {
        super(mensaje);
    }
}

```

ReglaNegocioException.java

```

package com.utp.tienda.exception;

public class ReglaNegocioException extends RuntimeException {
    public ReglaNegocioException(String mensaje) {
        super(mensaje);
    }
}

```

9. Paso 6 - Implementar la capa Service con @Transactional

Este es el núcleo de la práctica. Las consultas de solo lectura se marcan como readOnly = true. La operación registrarSalida se ejecuta dentro de una transacción de escritura.

src/main/java/com/utp/tienda/service/ProductoService.java

```

package com.utp.tienda.service;

```

```

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.List;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.utp.tienda.exception.RecursoNoEncontradoException;
import com.utp.tienda.exception.ReglaNegocioException;
import com.utp.tienda.model.MovimientoStock;
import com.utp.tienda.model.Producto;
import com.utp.tienda.repository.MovimientoStockRepository;
import com.utp.tienda.repository.ProductoRepository;

@Service
public class ProductoService {

    private final ProductoRepository productoRepository;
    private final MovimientoStockRepository movimientoRepository;

    public ProductoService(ProductoRepository productoRepository,
                           MovimientoStockRepository movimientoRepository) {
        this.productoRepository = productoRepository;
        this.movimientoRepository = movimientoRepository;
    }

    @Transactional(readOnly = true)
    public List<Producto> buscarPorNombre(String texto) {
        return productoRepository.buscarPorNombre(texto);
    }

    @Transactional(readOnly = true)
    public List<Producto> buscarPorCategoria(String categoria) {
        return productoRepository.findByCategoriaIgnoreCase(categoria);
    }

    @Transactional(readOnly = true)
    public List<Producto> buscarPorRango(BigDecimal min, BigDecimal max) {
        if (min.compareTo(max) > 0) {
            throw new ReglaNegocioException("El precio mínimo no puede superar al máximo");
        }
        return productoRepository.buscarPorRangoPrecio(min, max);
    }

    @Transactional
    public Producto registrarSalida(Long productoId, int cantidad) {
        Producto producto = obtenerProducto(productoId);
        validarSalida(producto, cantidad);

        producto.setStock(producto.getStock() - cantidad);

        MovimientoStock movimiento = new MovimientoStock(
            producto, "SALIDA", cantidad, LocalDateTime.now());
        movimientoRepository.save(movimiento);

        // El producto se actualiza mediante dirty checking al hacer commit.
        return producto;
    }

    @Transactional
    public void simularSalidaConError(Long productoId, int cantidad) {
        Producto producto = obtenerProducto(productoId);
        validarSalida(producto, cantidad);

        producto.setStock(producto.getStock() - cantidad);
        movimientoRepository.save(new MovimientoStock(
            producto, "SALIDA", cantidad, LocalDateTime.now()));

        throw new IllegalStateException(
            "Error simulado: la transacción debe hacer rollback");
    }

    private Producto obtenerProducto(Long id) {
        return productoRepository.findById(id)
            .orElseThrow(() -> new RecursoNoEncontradoException(
                "Producto no encontrado: " + id));
    }
}

```

```

    }

    private void validarSalida(Producto producto, int cantidad) {
        if (cantidad <= 0) {
            throw new ReglaNegocioException("La cantidad debe ser mayor que cero");
        }
        if (producto.getStock() < cantidad) {
            throw new ReglaNegocioException(
                "Stock insuficiente. Disponible: " + producto.getStock());
        }
    }
}

```

OBSERVA EL DISEÑO

ProductoService posee dependencias inmutables (final), usa inyección por constructor y no guarda estado de usuarios. La transacción se limita al método de negocio, no al Controller.

10. Paso 7 - Exponer los endpoints REST

src/main/java/com/utp/tienda/controller/ProductoController.java

```

package com.utp.tienda.controller;

import java.math.BigDecimal;
import java.util.List;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.utp.tienda.model.Producto;
import com.utp.tienda.service.ProductoService;

@RestController
@RequestMapping("/api/productos")
public class ProductoController {

    private final ProductoService service;

    public ProductoController(ProductoService service) {
        this.service = service;
    }

    @GetMapping("/buscar")
    public List<Producto> buscarPorNombre(@RequestParam String texto) {
        return service.buscarPorNombre(texto);
    }

    @GetMapping("/categoria/{categoria}")
    public List<Producto> buscarPorCategoria(@PathVariable String categoria) {
        return service.buscarPorCategoria(categoria);
    }

    @GetMapping("/precio")
    public List<Producto> buscarPorRango(
        @RequestParam BigDecimal min,
        @RequestParam BigDecimal max) {
        return service.buscarPorRango(min, max);
    }

    @PostMapping("/{id}/salidas")
    public ResponseEntity<Producto> registrarSalida(
        @PathVariable Long id,
        @RequestParam int cantidad) {
        return ResponseEntity.ok(service.registrarSalida(id, cantidad));
    }

    @PostMapping("/{id}/salidas/simular-error")
    public ResponseEntity<Void> simularError(
        @PathVariable Long id,
        @RequestParam int cantidad) {
        service.simularSalidaConError(id, cantidad);
        return ResponseEntity.noContent().build();
    }
}

```

```
}
}
```

11. Paso 8 - Manejo global de errores

El backend debe responder con códigos HTTP coherentes. Para el laboratorio usaremos una respuesta JSON mínima con marca de tiempo, estado y mensaje.

src/main/java/com/utp/tienda/exception/ApiExceptionHandler.java

```
package com.utp.tienda.exception;

import java.time.LocalDateTime;
import java.util.Map;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(RecursoNoEncontradoException.class)
    public ResponseEntity<Map<String, Object>> noEncontrado(
        RecursoNoEncontradoException ex) {
        return respuesta(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(ReglaNegocioException.class)
    public ResponseEntity<Map<String, Object>> reglaNegocio(
        ReglaNegocioException ex) {
        return respuesta(HttpStatus.BAD_REQUEST, ex.getMessage());
    }

    @ExceptionHandler(IllegalStateException.class)
    public ResponseEntity<Map<String, Object>> errorSimulado(
        IllegalStateException ex) {
        return respuesta(HttpStatus.INTERNAL_SERVER_ERROR, ex.getMessage());
    }

    private ResponseEntity<Map<String, Object>> respuesta(
        HttpStatus status, String mensaje) {
        return ResponseEntity.status(status).body(Map.of(
            "timestamp", LocalDateTime.now().toString(),
            "status", status.value(),
            "error", status.getReasonPhrase(),
            "message", mensaje));
    }
}
```

12. Paso 9 - Preparar datos de prueba

Use el CRUD de la Semana 6 o inserte productos desde MySQL. Ejemplo:

Datos de ejemplo

```
INSERT INTO productos (nombre, categoria, precio, stock) VALUES
('Laptop Lenovo ThinkPad', 'Tecnologia', 4200.00, 8),
('Mouse Logitech MX', 'Tecnologia', 320.00, 20),
('Silla ergonómica', 'Muebles', 850.00, 6),
('Escritorio ejecutivo', 'Muebles', 1200.00, 4),
('Monitor 27 pulgadas', 'Tecnologia', 1450.00, 10);
```

13. Paso 10 - Ejecutar y probar las consultas JPQL

Inicia la aplicación desde Visual Studio Code o con Maven:

```
mvn spring-boot:run
```

13.1. Búsqueda por nombre

```
GET http://localhost:8080/api/productos/buscar?texto=lap
```

Resultado esperado: se devuelve la Laptop Lenovo ThinkPad. Observe en la consola el SQL generado por Hibernate.

13.2. Consulta derivada por categoría

```
GET http://localhost:8080/api/productos/categoria/Tecnologia
```

Resultado esperado: todos los productos de la categoría, sin depender de mayúsculas/minúsculas.

13.3. Consulta JPQL por rango de precio

```
GET http://localhost:8080/api/productos/precio?min=300&max=1500
```

Resultado esperado: productos cuyo precio se encuentra entre 300 y 1500, ordenados de menor a mayor precio.

PRUEBA NEGATIVA

Ejecuta /api/productos/precio?min=2000&max=100. La API debe responder 400 porque la regla de negocio rechaza un rango invertido.

14. Paso 11 - Comprobar una transacción exitosa

14.1. Consultar el stock inicial

```
SELECT id, nombre, stock
FROM productos
WHERE id = 1;
```

14.2. Registrar una salida desde Postman

```
POST http://localhost:8080/api/productos/1/salidas?cantidad=2
```

Si el stock inicial era 8, la respuesta debe mostrar stock 6. Luego valide en MySQL:

```
SELECT id, nombre, stock FROM productos WHERE id = 1;

SELECT id, producto_id, tipo, cantidad, fecha
FROM movimientos_stock
ORDER BY id DESC;
```

¿QUÉ OCURRIÓ?

La misma transacción modificó el stock y registró el movimiento. Al finalizar sin errores, Spring realizó el commit.

15. Paso 12 - Comprobar rollback con un error intencional

El objetivo es demostrar que el cambio no queda “a medias”. Primero anote el stock y la cantidad de movimientos existentes.

```
SELECT stock FROM productos WHERE id = 1;
SELECT COUNT(*) AS movimientos FROM movimientos_stock WHERE producto_id = 1;
```

Ahora ejecute:

Java 25 • Spring Boot • Maven • MySQL • Visual Studio Code

```
POST http://localhost:8080/api/productos/1/salidas/simular-error?cantidad=1
```

La API responderá 500 porque el método lanza una `IllegalStateException`. Vuelva a ejecutar las consultas SQL anteriores.

RESULTADO ESPERADO

El stock y el número de movimientos deben permanecer exactamente como estaban antes de la llamada. La excepción provocó rollback de toda la transacción.

16. Qué debe comprender el estudiante después del experimento

16.1. @Transactional no “guarda” por sí solo

La anotación delimita una unidad de trabajo. Dentro de ella, Spring coordina el EntityManager y la transacción del recurso. Hibernate detecta cambios en entidades administradas y sincroniza el estado con la base de datos al momento apropiado.

16.2. readOnly = true expresa intención

En métodos que solo consultan datos, `@Transactional(readOnly = true)` comunica que la operación es de lectura. Dependiendo del proveedor y la configuración, puede habilitar optimizaciones; sobre todo, mejora la claridad del diseño.

16.3. No colocar @Transactional indiscriminadamente

- Evite usar `@Transactional` en el Controller: la frontera de negocio pertenece al Service.
- Evite transacciones excesivamente largas que incluyan lógica ajena a la base de datos.
- Recuerde que una llamada interna a otro método del mismo objeto no atraviesa necesariamente el proxy transaccional de Spring.
- No capture y “silencie” una excepción si espera rollback; si el error no se propaga, Spring puede confirmar la transacción.
- Las consultas masivas de actualización (`@Modifying`) pueden saltarse el dirty checking habitual y requieren más cuidado con el contexto de persistencia.

16.4. Errores frecuentes y diagnóstico

Síntoma	Causa probable	Qué revisar
JPQL falla al iniciar	Se usó nombre de tabla/columna en lugar de entidad/atributo.	Producto, p.precio, p.nombre.
400 Bad Request	Parámetro con formato incompatible.	min, max, cantidad y tipos Java.
404	ID inexistente o ruta incorrecta.	PathVariable y datos en MySQL.
Rollback no ocurre	La excepción fue capturada o no activa rollback.	Propagación de RuntimeException.
LazyInitializationException	Se intenta acceder a relación LAZY fuera del contexto.	No serializar entidades indiscriminadamente; usar DTO cuando corresponda.
Datos no cambian	No se inició transacción o entidad no está administrada.	Ubicación de <code>@Transactional</code> y flujo Service → Repository.

17. Actividad de consolidación

Implemente las siguientes mejoras sin alterar la separación por capas:

1. Endpoint GET `/api/productos/stock-bajo?limite=5` usando `buscarConStockBajo()`.
2. Validar que texto de búsqueda no sea vacío ni contenga solo espacios.

3. Crear una operación transaccional registrarEntrada(id, cantidad) que incremente stock y registre un movimiento de tipo ENTRADA.
4. Probar una entrada exitosa y una salida con stock insuficiente.
5. Documentar, con evidencia de MySQL, qué operaciones realizan commit y cuáles rollback.

17.1. Reto avanzado opcional

Investigue cómo se implementaría una consulta de actualización masiva con @Modifying + @Query. Explique por qué una actualización masiva JPQL debe manejarse con cuidado respecto del contexto de persistencia. No es obligatorio implementarla en el flujo principal.

18. Preguntas de reflexión

1. ¿Por qué JPQL utiliza el nombre de la entidad y no el nombre de la tabla?
2. ¿Cuándo elegirías una consulta derivada y cuándo @Query?
3. ¿Por qué la frontera transaccional se ubica normalmente en la capa Service?
4. ¿Qué diferencia existe entre llamar save() y confiar en dirty checking dentro de una transacción?
5. ¿Qué ocurriría si el stock se descuenta pero el registro del movimiento falla y no existiera una transacción?
6. ¿Qué tipo de excepción provoca rollback por defecto en Spring?
7. ¿Por qué una transacción no debería mantenerse abierta mientras se realiza una llamada remota lenta?
8. ¿Qué problema podría aparecer si devolvemos directamente entidades JPA con relaciones LAZY como respuesta REST?

19. Síntesis de la Semana 7

DEL CRUD A LA LÓGICA TRANSACCIONAL

En la Semana 6 el estudiante aprendió a persistir entidades y ejecutar CRUD. En la Semana 7 avanza hacia consultas expresivas con JPQL y operaciones de negocio consistentes mediante transacciones. Esta base prepara el camino para incorporar seguridad y autenticación en las siguientes semanas.

Concepto	Qué debe quedar claro
JPQL	Consulta entidades y atributos Java; Hibernate traduce a SQL.
@Query	Permite expresar consultas personalizadas en el repositorio.
@Transactional	Delimita una unidad de trabajo en la capa de servicio.
Dirty checking	Hibernate detecta cambios en entidades administradas.
Commit	Confirma todos los cambios de la transacción.
Rollback	Revierte todos los cambios cuando la transacción falla.

19.1. Checklist final del estudiante

- La aplicación inicia sin errores y se conecta a MySQL.
- Las consultas JPQL devuelven los resultados esperados.
- La operación de salida descuenta stock y crea un movimiento.
- El endpoint de error simulado no deja cambios parciales.
- Puedo explicar con mis propias palabras por qué ocurrió el rollback.
- Comprendo qué responsabilidad tiene Controller, Service y Repository.